

軟體工程導論 (Introduction to Software Engineering)

講義大綱與核心概念

- 1.1 技術演進脈絡：從機械化、數位革命到普及化人工智慧 (Industry 1.0~4.0)。
- 1.2 軟體工程起源與「軟體危機」：1968 年 NATO Garmisch 會議為何宣告軟體開發必須從個人工藝走向工程紀律。
- 1.3 軟體本質剖析：軟體為何遠超過程式碼 (程式、數據、SOP、文檔)。
- 1.4 ISO 9126 品質模型：6 大特徵與關鍵子屬性實務案例圖解。
- 1.5 現代軟體多元樣貌：從雲端 SaaS、行動應用、企業 ERP 到嵌入式物聯網與 AI 平台。
- 1.6 何謂軟體工程？：正式定義、軟體工程流程 (SE Process)、約束與資源權衡天平、四大通用核心活動、工程多元維度與迷思破解。
- 1.7 現代工具鏈與 AI 雙面刃：CI/CD 自動化流程與 AI 在生命週期各階段的效益與風險矩陣。
- 1.8 專業倫理、社會責任與欺騙性黑暗模式：ACM/IEEE 倫理守則、重大歷史災難與 UI/UX 黑暗模式純黑白四格漫畫圖解。
- 1.9 軟體工程常見問答 (FAQ)。
- 附錄：互動活動解答與詳細解析。

投影片連結：[Ch1 軟體工程導論](#)



1.1 技術演進脈絡：從機械化到普及化 AI

要理解軟體工程為何成為一門正式學科，必須先回顧過去兩個世紀人類生產力的演進歷程。軟體並非憑空誕生，而是歷經四次工業革命浪潮，逐步成為現代人類社會運轉的中樞神經。

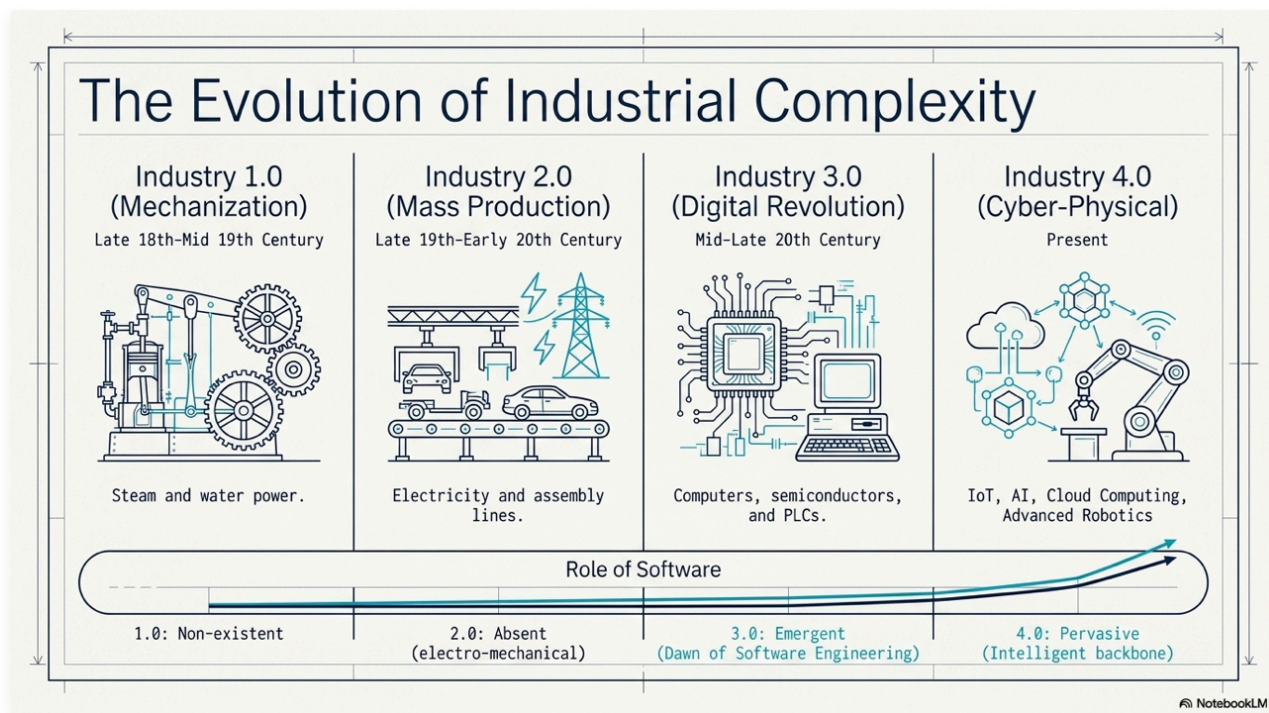


圖 1.1：四次工業革命歷程——生產力驅動力從人體肌肉、機械能量、數位邏輯逐步轉移至自主認知智慧。

1.1.1 四大工業革命時代

- 工業 1.0 —— 機械化時代（18 世紀末至 19 世紀中葉）** 以蒸汽機與水力動力取代人體與牲畜肌肉，工廠制度應運而生，深刻改變了紡織業、煤礦開採與交通運輸（蒸汽火車與輪船）。此時代科技純屬機械與物理範疇，完全沒有「軟體」的概念。
- 工業 2.0 —— 大規模量產時代（19 世紀末至 20 世紀初）** 電力、內燃機與亨利·福特的流水線裝配作業，推動了消費品、汽車與電器的標準化大規模生產。雖然電磁繼電器與電話網路奠定了自動通訊的早期基礎，但控制邏輯仍純粹由物理硬體電路硬焊決定。
- 工業 3.0 —— 數位革命與軟體萌芽（20 世紀中葉至末期）** 半導體、微處理器與大型主機的發明引領人類進入數位時代。程式化邏輯控制器 (PLC) 自動化了工業機台，個人電腦 (PC) 走入企業與家庭。在這一時期，軟體正式獨立成為一門專業領域：人們開始撰寫程式來控制硬體、處理金融數據與路由通訊封包。
- 工業 4.0 —— 互聯、雲端與 AI 時代（21 世紀初至今）** 虛實整合系統 (CPS)、物聯網 (IoT)、雲端運算與人工智慧深度融合。軟體不再只是輔助業務的工具，而是支撐醫療、交通、金融與國家治理的基礎骨幹。現代軟體具備高度互聯、自我調適與無所不在的特質。

💡 課堂互動活動 —— 隨堂投票與討論：

- **投票：**檢視你每天使用的各項數位服務與裝置，你認為其中有多少比例屬於工業 3.0 確定性邏輯，又有多少比例屬於工業 4.0 的自適應 AI 聯網軟體？
- **討論：**請列舉一個你認為急需進行「工業 4.0 軟體轉型」的傳統流程（例如：校園停車管理、急診檢傷分類、大眾運輸排班）。在轉型過程中會面臨哪些獨特的軟體工程挑戰？

1.2 軟體工程的起源與「軟體危機」

在計算機發展初期（1940～1950 年代），硬體極為昂貴且稀少，軟體僅被視為附屬於特定硬體的指令集。程式規模很小，通常由少數科學家以組合語言或機器碼編寫，開發過程猶如個人工藝手作。

然而，隨著 1960 年代硬體運算能力呈指數級提升、成本大幅下降，企業與國防機構開始委託開發超大型複雜系統——如軍事防空雷達 (SAGE)、民航訂位系統 (SABRE) 與太空船導航計算機。

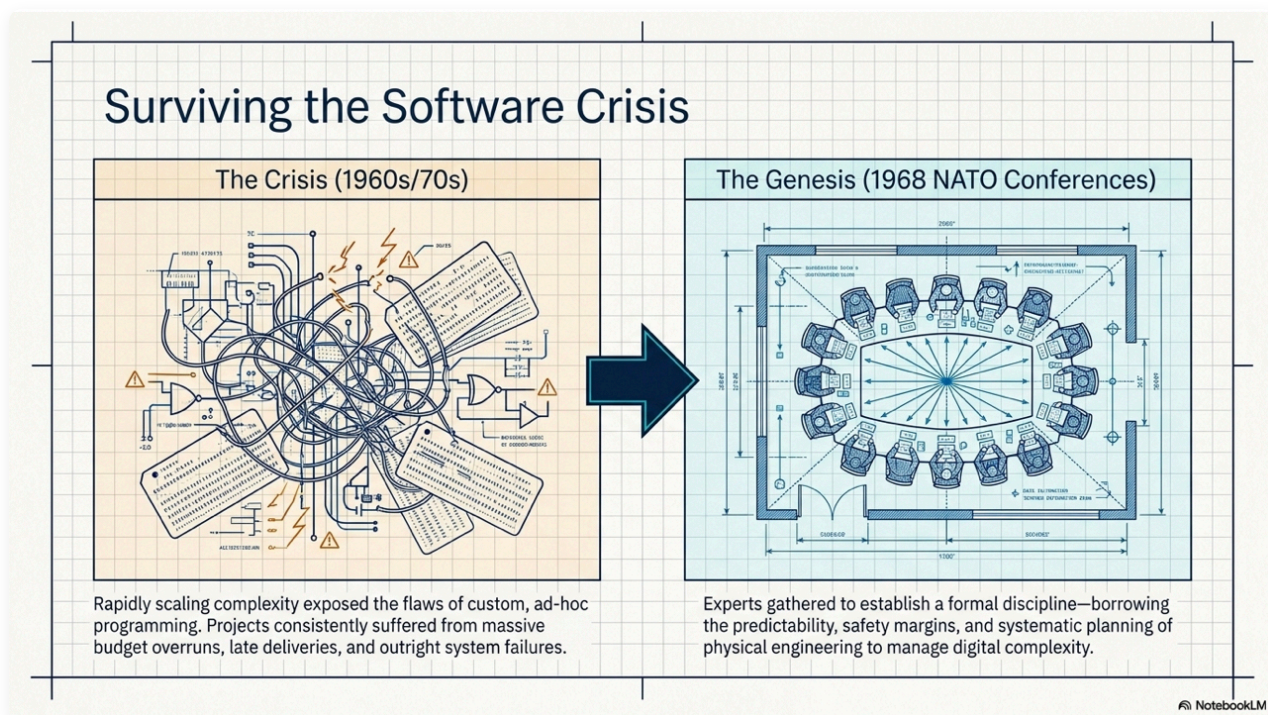


圖 1.2.1：1960 年代後期的軟體危機——專案嚴重超支、交付延宕、缺陷頻傳，且上線後難以維護。

1.2.1 1960 年代後期的軟體危機

到了 1960 年代後期，軟體產業撞上了嚴重的技術高牆，史稱**軟體危機 (Software Crisis)**：

- **成本嚴重超支：**最終花費經常達到最初預算的 3 到 4 倍。
- **交付嚴重延宕：**專案比預期晚數年交付，甚至在完成前即遭腰斬廢棄。
- **品質缺陷頻傳：**系統頻繁當機、運算結果錯誤，甚至引發嚴重的公共安全隱患。

- **系統無法維護**：缺乏模組化架構與文件說明的程式碼，一旦修改便引發連鎖崩潰。

危機的核心根源在於：將編寫程式碼視為孤立的手工藝做法，完全無法應對跨團隊、長週期、高複雜度的現代大型軟體系統。

1.2.2 1968 年 NATO Garmisch 會議：學科的誕生

為了徹底解決此一危機，北約科學委員會於 1968 年 10 月在德國加米施 (Garmisch) 召開歷史性會議，匯聚了 50 位頂尖電腦科學家、業界主管與學者。



圖 1.2.2：1968 年歷史性的 NATO 軟體工程會議，正式確立了「軟體工程」一詞，宣告將工程嚴謹度引入軟體開發。

會議正式確立了 **軟體工程 (Software Engineering)** 這一術語，其核心目標是：推動軟體開發從隨意無序的手工編程，轉型為具備結構化方法論、形式化規格、成本估算、嚴格驗證與專案管理的正式工程學科。

1.2.3 軟體工程失敗的真實代價

缺乏工程紀律的後果絕不僅僅是金錢損失，更可能造成災難性的生命危害：

- **名古屋空難 (1994)**：空中巴士 A300 由於自動駕駛與飛行員手動操作產生人機介面 (HMI) 軟體邏輯衝突，導致 264 人不幸喪生。
- **火星氣候探測器 (1999)**：價值 3.27 億美元的探測器在進入火星大氣層時墜毀，原因僅是兩個模組間的力學單位未經校驗（公制牛頓 vs. 英制磅力）。

- 阿利安 5 號火箭 501 航班 (1996)：發射 37 秒後自行引爆解體，損失逾 3.7 億美元，根源為 64 位元浮點數轉為 16 位元有號整數時未做溢位例外處理。

💡 🧠 觀念檢驗題 (CCQ 1) —— 軟體危機的本質：

問題：為何 1968 年的軟體危機無法僅透過採購運算速度更快的超級電腦或擴充硬體記憶體來解決？

- A) 1960 年代後期硬體製造技術停滯不前。
- B) 危機本質上是人類心智面對架構複雜度、跨人溝通成本與缺乏規範的認知危機，更快的硬體只會放大問題規模。
- C) 當時的程式語言缺乏數學計算能力。
- D) 當時的硬體無法連接雲端架構。

👉 [前往附錄查看解答與詳細解析](#)

1.3 軟體本質剖析：超越原始碼的四大支柱

初學者常誤以為軟體就是原始程式碼 (Source Code) 的代名詞。然而在專業軟體工程中，原始碼僅僅是冰山浮出水面的一角。

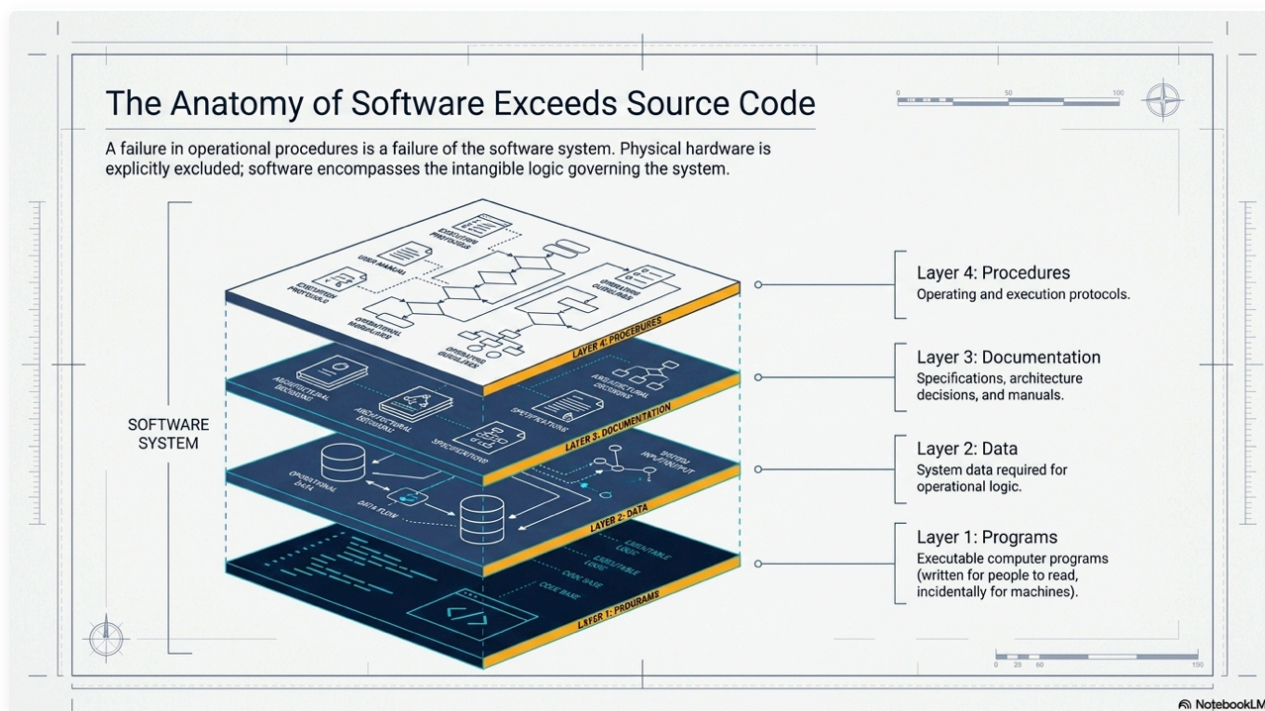


圖 1.3：專業軟體的解剖結構——由執行程式、資料資產、維運程序與架構文檔共同構成。

1.3.1 IEEE 的軟體標準定義

IEEE 標準對軟體的正式定義為：

軟體 (Software)：與計算機系統運作相關的電腦程式、程序、以及可能伴隨的相關文件與資料。

1.3.2 四大核心支柱：程式、資料、程序與文檔

根據此定義，專業軟體系統包含四大核心支柱：

1. **Programs (程式碼與二進制執行檔)：**以 Python、Java、C++ 或 TypeScript 撰寫、指示 CPU 執行的邏輯。
2. **Data & Schemas (資料與綱要)：**資料庫表格、配置檔案、初始化種子資料集與 AI 模型權重檔。
3. **Operational Procedures (維運程序與 SOP)：**部署腳本、備份例程、災難復原演練守則與 CI/CD 自動化腳本。
4. **Documentation (架構與規格文檔)：**架構決策記錄 (ADR)、OpenAPI/Swagger 介面規格書、需求規格書 (SRS) 與使用者手冊。

誠如電腦科學家 Harold Abelson 名言：

「程式寫出來主要是給人閱讀的，順便給機器執行而已。」

雙人分組活動 —— 實務中的軟體冰山：

- 與隔壁同學分組，挑選一項日常熱門服務：Google Maps、Uber 或 Spotify。
- 兩人合作針對四大支柱（程式、資料、程序、文檔）各列出一項具體產出物。
- 深入探討：如果一個工程團隊遺失了所有部署腳本與資料庫綱要，他們能否單憑原始程式碼在產線上順利重建並維運業務？為什麼？

1.4 何謂「好」軟體？ISO 9126 品質模型

軟體品質具有多維度特性。國際標準組織制定了 ISO/IEC 9126 品質模型，將軟體品質劃分為 6 大特徵與多個可測試的子屬性。

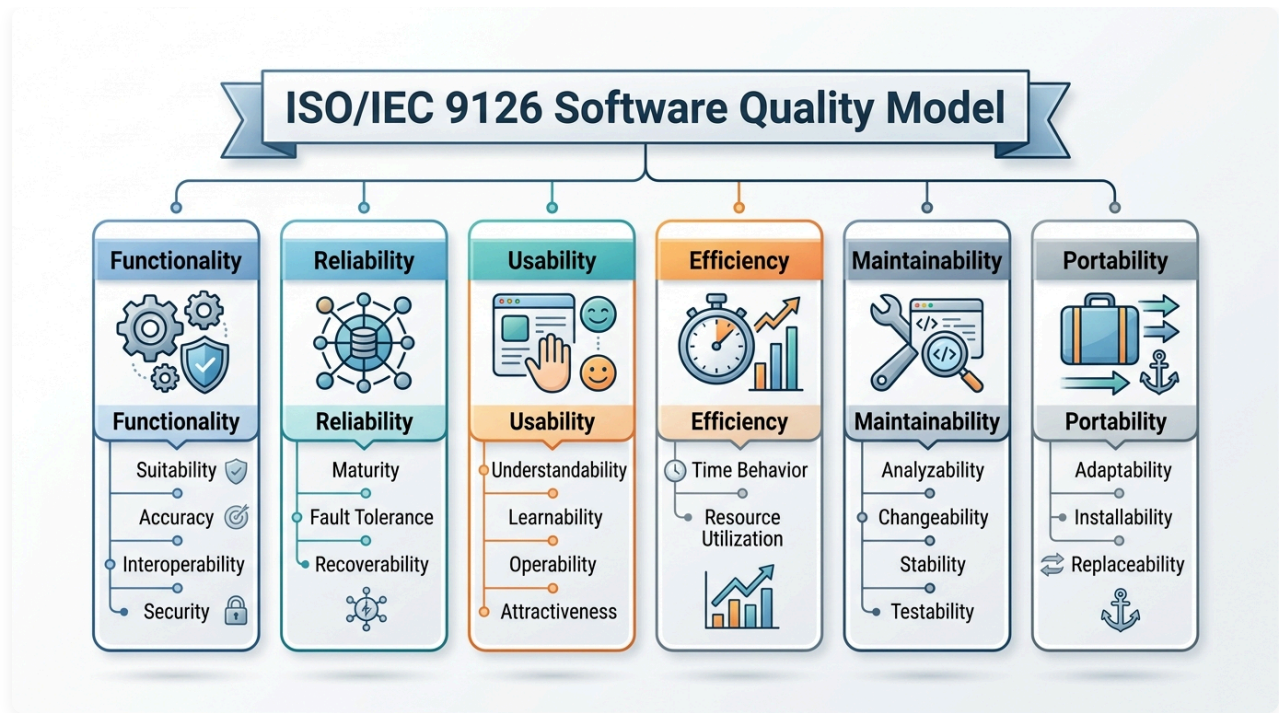


圖 1.4：ISO 9126 軟體品質 6 大特徵與關鍵子屬性完整架構圖解。

1.4.1 ISO 9126 六大特徵與子屬性深度解析

1. Functionality (功能性)

軟體滿足明確指定與隱含業務及技術需求的能力。

- **Suitability (適用性)**：系統是否提供符合預期任務的完整功能？（如電商支援購物車、折扣券與多幣別結帳）。
- **Accuracy (準確性)**：運算結果是否精確無誤？（如財務會計引擎計算扣稅額達零分位漂移）。
- **Interoperability (互操作性)**：能否與外部系統順暢交換數據？（如醫療系統透過標準 HL7/FHIR API 傳輸病歷）。
- **Security (資安性)**：能否防範未授權存取與攻擊？（如強制 MFA、RBAC 權限控管與 AES-256 資料庫加密）。
- **Functionality Compliance (功能合規性)**：符合特定法規或產業功能標準。

2. Reliability (可靠性)

軟體在指定條件與時間內維持效能水準的能力。

- **Maturity (成熟度)**：在長期運作中因潛在 Bug 導致當機的頻率極低（平均故障間隔時間 MTBF 極高）。
- **Fault Tolerance (容錯性)**：面臨非預期例外或第三方服務中斷時，能否優雅降級而不崩潰？（如主金流超時自動切換備援金流）。
- **Recoverability (可復原性)**：遭遇崩潰後能否迅速恢復運作狀態並回復數據？（如資料庫崩潰後 30 秒內透過 WAL 復原一致性）。
- **Reliability Compliance (可靠性合規性)**：達成 99.99% 高可用性服務水準合約 (SLA)。

3. Usability (易用性)

使用者理解、學習、操作與喜愛該軟體所需付出的心力。

- **Understandability (易理解性)**：新手能否在介面上一目了然各項功能的作用？
- **Learnability (易學習性)**：使用者掌握核心操作的速度（如新進護理師經 15 分鐘教學即可流暢操作 EHR 系統）。
- **Operability (易操作性)**：日常工作流程是否流暢無阻（包含鍵盤快捷鍵、無障礙 WCAG 2.1 輔助支援）。
- **Attractiveness (介面吸引力)**：視覺排版協調、間距舒適、不易引起視覺疲勞。

4. Efficiency (效率性 / 效能)

軟體效能水準與所消耗運算資源之間的比例關係。

- **Time Behavior (時間行為)**：尖峰負載下的回應延遲與交易吞吐量（如搜尋 API 在 99% 的請求中達成延遲小於 80ms）。
- **Resource Utilization (資源利用率)**：CPU、記憶體與頻寬的消耗效率（如單節點以低於 250MB 記憶體承載 10,000 條 WebSocket 連線）。

5. Maintainability (可維護性)

修改軟體（包含修復 Bug、效能調校與新增功能）所需付出的成本與難易度。

- **Analyzability (可分析性)**：工程師定位錯誤根因的容易程度（如透過 OpenTelemetry 分散式追蹤在數分鐘內鎖定出錯的微服務）。
- **Changeability (可變更性)**：修改特定模組時不必重寫大量無關代碼。
- **Stability (穩定性)**：引入新功能時不易在舊系統中引發意料之外的副作用。
- **Testability (可測試性)**：代碼架構是否便於編寫單元測試與自動化迴歸測試（如採用依賴注入讓外部 API 輕鬆被 Mock）。

6. Portability (可移植性)

軟體從某一硬體、作業系統或雲端環境遷移至另一環境的難易度。

- **Adaptability (適應性)**：無需修改代碼即可跨 Linux x86、macOS ARM64 與 Windows 執行。
 - **Installability (可安裝性)**：部署流程是否簡易快速（如透過 `docker compose up` 60 秒建置完整環境）。
 - **Co-existence (共存性)**：能否在同一主機上與其他軟體和諧運行而不產生相依套件衝突。
 - **Replaceability (可替換性)**：更換底層組件的容易程度（如透過 ORM 介面將 MySQL 替換為 PostgreSQL）。
-

💡 🇮🇹 課堂互動活動 —— 品質屬性權衡投票：

- **情境：**在真實工程中，受限於預算與效能，無法同時將所有品質維度最大化。
- **投票：**請為以下兩個系統挑選出最關鍵、不可妥協的「前兩大」ISO 9126 品質屬性：
 - i. 加護病房自動胰島素注射幫浦控制軟體。
 - ii. 熱門休閒手機小遊戲。
- **討論：**為何在醫療設備中將「上市速度 (Time to Market)」置於「容錯性 (Fault Tolerance)」之上是致命的，而休閒遊戲卻可適度容忍非致命錯誤以換取快速迭代？

1.5 現代軟體的多元樣貌

現代軟體部署於極為多元的運行環境中，各自具備完全不同的架構模式、測試方法與維運約束。

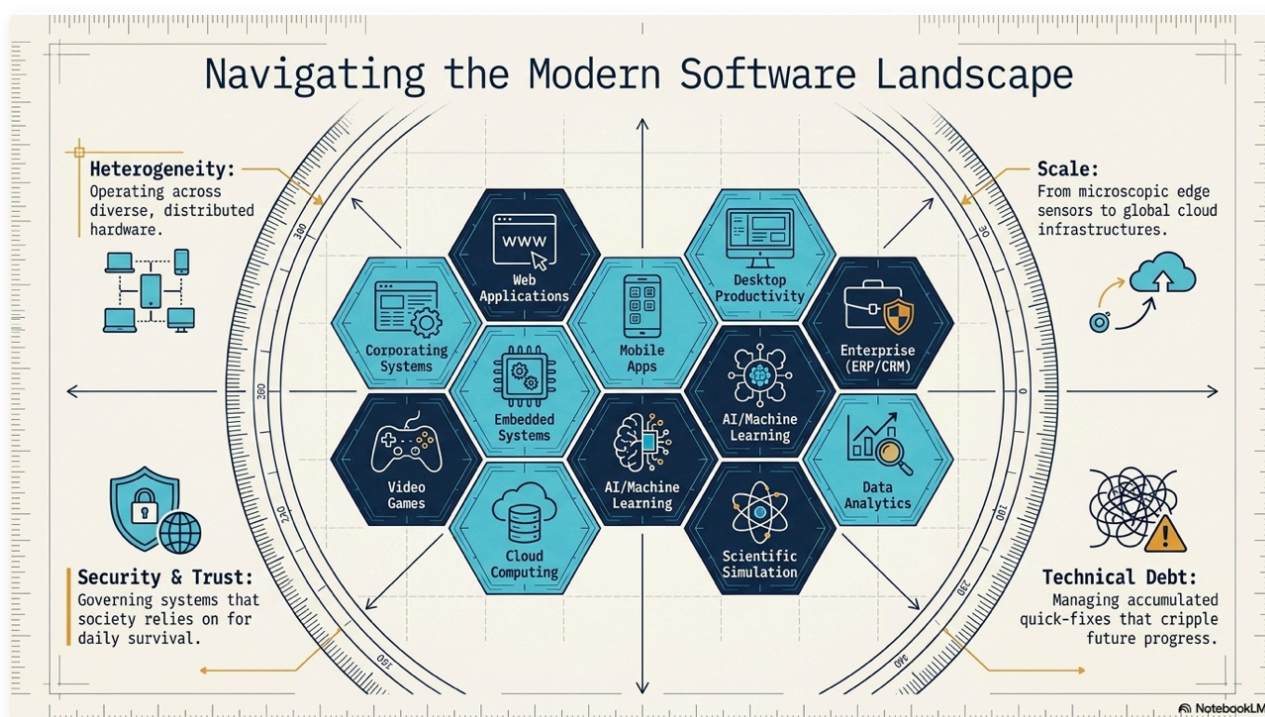


圖 1.5：現代軟體多元樣貌——涵蓋雲端原生 Web 應用、行動 App、企業 ERP、嵌入式物聯網與 AI 平台。

1.5.1 主要應用領域

- **Web 與 SaaS 雲端平台：**分散式彈性架構、高可用性、微服務、零停機部署（如 Slack、Netflix、Salesforce）。
- **行動應用程式 (Mobile Apps)：**受限的電池電量、觸控互動、不穩定的網路連線（如 iOS / Android 原生與跨平台應用）。
- **企業級核心系統 (ERP / CRM)：**支援複雜業務流程、嚴格 ACID 資料庫交易與權限審計（如 SAP、Oracle）。

- **嵌入式韌體與物聯網 (IoT)**：嚴苛的即時性、極限記憶體限制、攸關生死的車載與航電系統。
- **人工智慧與機器學習 (AI / ML)**：資料密集型管線、向量資料庫、GPU 運算加速與大語言模型推理。
- **科學運算與 CAD 軟體**：嚴格的浮點數高精度運算、物理模擬與硬體圖形加速。

 課堂互動活動 —— 系統分類與混合架構探討：

- 以現代 **Tesla 自駕智慧電動車** 為例。
- 請分析它涵蓋了哪些軟體領域？（例如：底層煞車馬達即時嵌入式韌體、座艙中控觸控導航 UI、雲端車隊資料遙測微服務、邊緣端 FSD 自駕 AI 模型）。
- **討論**：為何各子系統的更新發布週期截然不同（如雲端微服務每天更新 vs. 煞車韌體極度嚴謹的 OTA 審查）？

1.6 何謂軟體工程？定義、流程、約束與核心活動

為什麼我們不能單純「把程式寫得更快更多」？為什麼我們需要「軟體工程」？

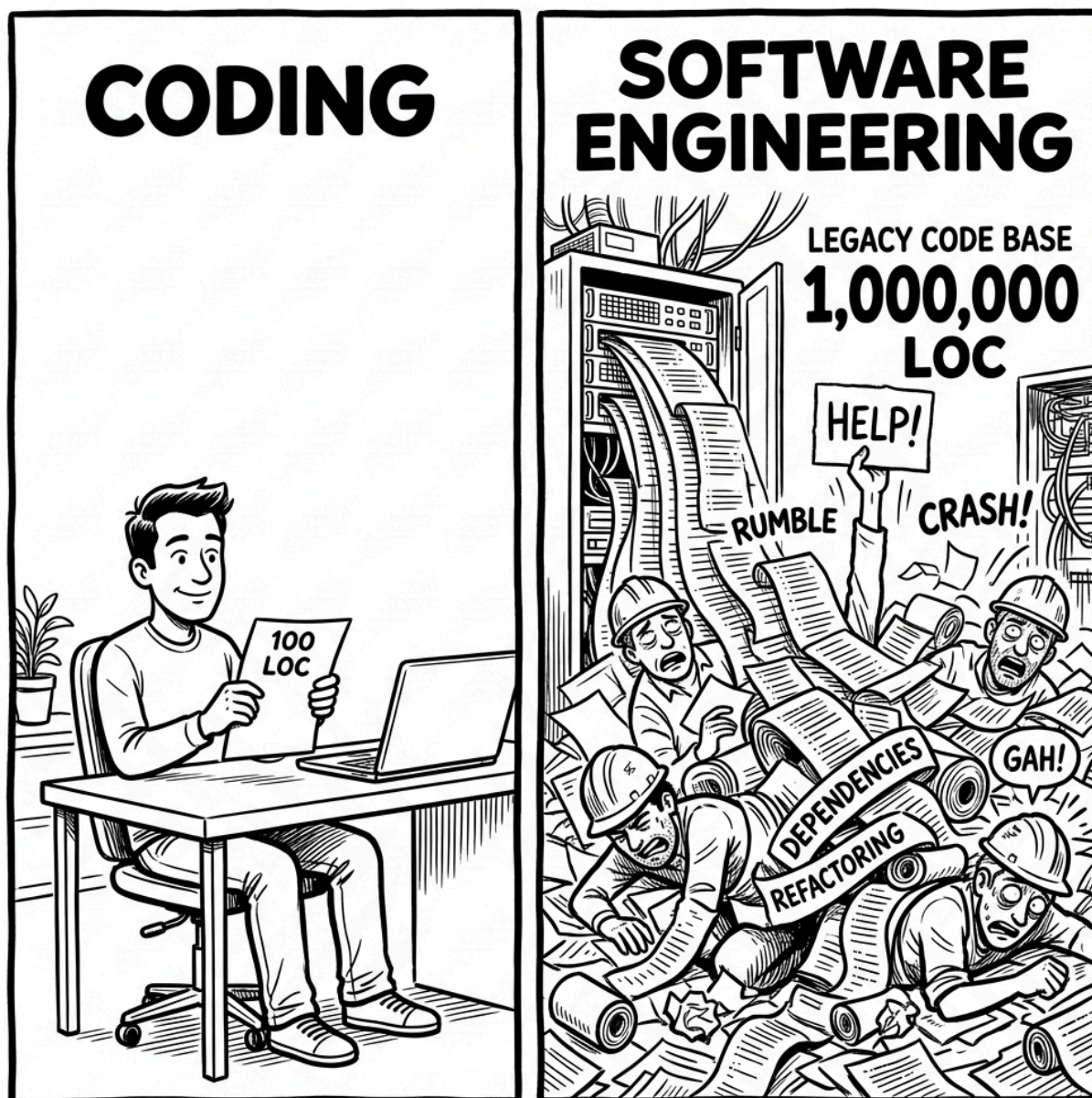


圖 1.6.1：小型腳本可由個人憑工藝手工打造，但百萬行規模的大型系統必須仰賴工程紀律才能長治久安。

1.6.1 軟體工程的正式定義與流程概念

軟體工程 (Software Engineering)：是一門工程學科，關注軟體生產的所有層面——從前期的系統需求規格定義，到系統上線後的維護與長期演進。

從這個經典定義出發，我們必須深入剖析兩大核心支柱：

1. **工程學科 (Engineering Discipline —— 在約束與資源下工作)**：工程既不是無拘無束的藝術創作，也不是純粹的理論科學；工程是在嚴苛的經濟、技術與法規約束 (Constraints) 下，善用有限的資源 (Resources) 來解決實際問題。

2. 軟體生產的所有層面 (The Software Engineering Process 軟體工程流程)：軟體開發絕不僅僅是撰寫代碼，它由一套系統化的軟體工程流程 (SE Process) 所驅動——包含有組織的活動、方法、角色與產出物，確保系統從概念成形、架構設計、編程、驗證到交付維運皆受到嚴密治理。

1.6.2 工程的本質：在約束中平衡有限資源

所有工程領域都有一個共通的本質：在嚴苛的約束 (Constraints) 下，善用有限的資源 (Resources) 來解決實際問題。

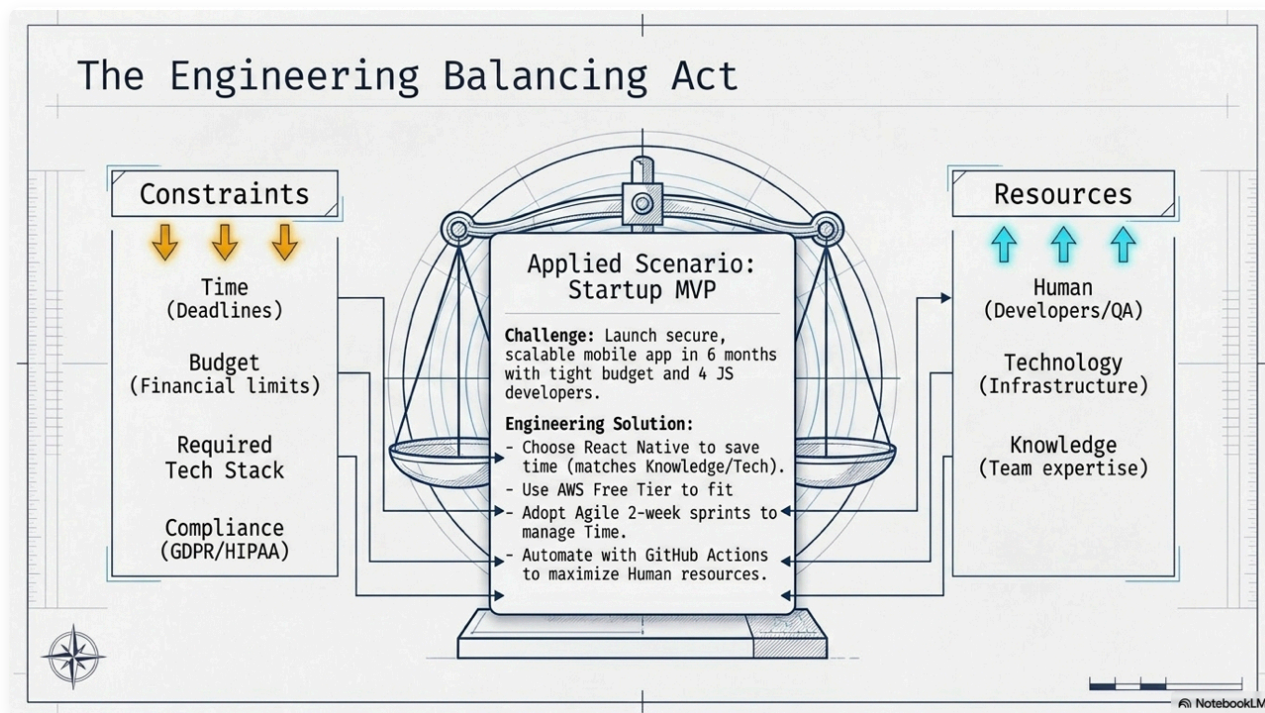


圖 1.6.2：軟體工程天平——在範疇、時間、預算與品質之間，動態調配人力、技術與基礎設施資源。

1. Constraints (現實約束)

- 時間約束 (Time)：專案死線 (Deadline)、市場搶先發布窗口。
- 預算約束 (Budget)：人力薪資上限、雲端運算費用配額、第三方工具授權費。
- 技術與平台約束 (Technology)：必須相容遺留資料庫、支援舊款手機 OS 版本。
- 法規與合規約束 (Regulations)：GDPR 個資隱私保護、HIPAA 醫療隱私法規、PCI-DSS 金流標準。
- 規模與吞吐量約束 (Scalability)：必須承載雙 11 尖峰每秒 50,000 筆訂單。

2. Resources (可用資源)

- 人力資源 (Human)：工程師技術專長、UI/UX 設計師、QA 測試員。

- **技術與基礎設施資源 (Tech & Infra)**：雲端配額 (AWS/GCP)、成熟開源函式庫、自動化 CI/CD 工具。
- **領域知識與經驗 (Knowledge)**：團隊對業務領域邏輯與設計模式的掌握度。

實務案例：醫療新創團隊 MVP 突圍戰

- **挑戰**：一家數位醫療新創團隊必須在 6 個月內以 8 萬美元的有限預算推出視訊問診 App，且必須通過 HIPAA 資安加密法規。
- **工程權衡解決方案**：
 - 範疇精簡 (約束管理)**：打造只具備視訊問診與處方預約的最小可行產品 (MVP)，將複雜的保險理賠整合推遲至第二期。
 - 跨平台技術選擇 (資源最佳化)**：採用 **Flutter** 建立單一跨平台代碼庫，同時發布 iOS 與 Android，節省近 40% 的前端人力。
 - 使用託管雲端服務 (基礎設施槓桿)**：採用符合 HIPAA 認證的託管 BaaS (Supabase / Firebase) 替代自建實體伺服器。
 - 全自動化 CI/CD (品質保證)**：建立 **GitHub Actions** 自動化測試與 Lint 檢查，讓 3 人小團隊無需龐大 QA 編制即可維持高代碼品質。

1.6.3 軟體工程流程的核心活動 (The Core Activities of SE Process)

在軟體工程中，無論團隊採用何種流程模型（瀑布模型、敏捷 Sprint、螺旋循環、持續 DevOps），核心皆由四大通用核心活動所組成：

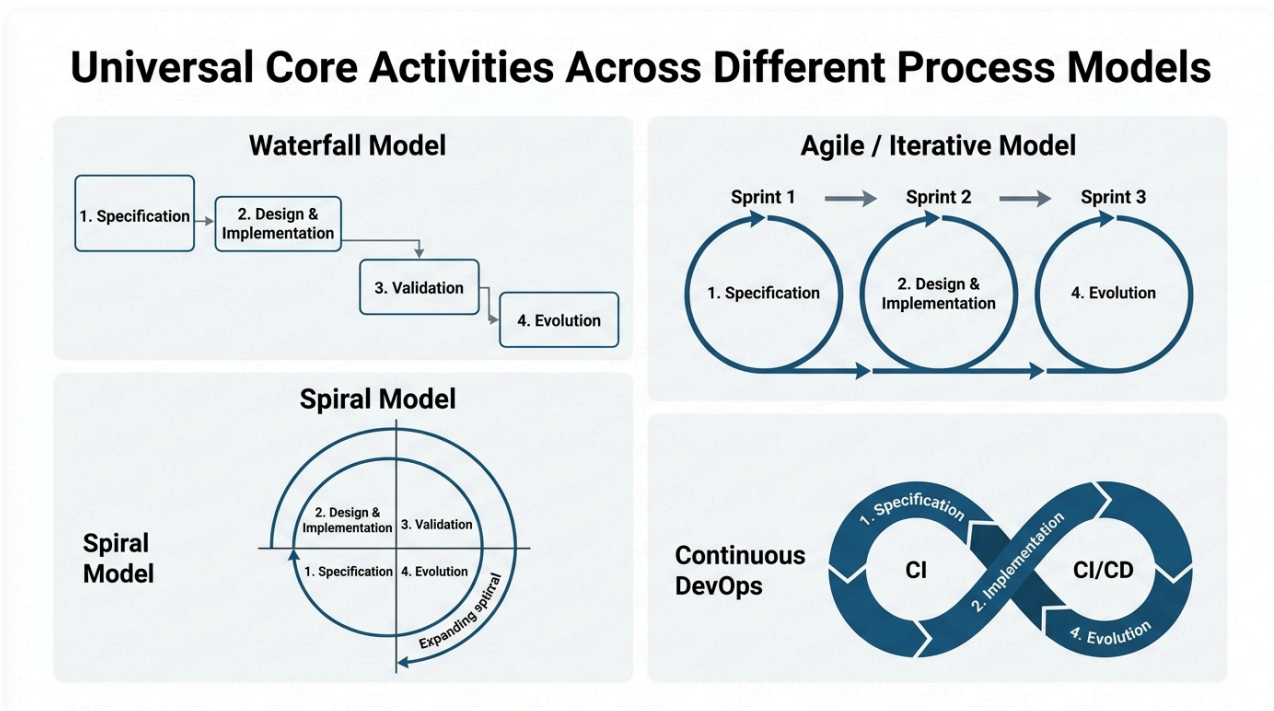


圖 1.6.3：四大通用核心活動（需求規格、設計與實作、驗證確認、系統演進）在各主流流程模型中的組織型態——從瀑布模型的線性串接、敏捷的衝刺循環，到螺旋模型的風險展開與 DevOps 的無限自動化迴圈。

四大核心活動深度解析對照表

核心活動	關鍵產出物 (Artifacts)	若忽視此活動的嚴重代價
1. 需求規格 (Specification)	使用者故事、需求規格書 (SRS)、使用案例圖	打造出完全無人使用的錯誤產品；驗收期重工成本暴增 100 倍
2. 設計與實作 (Design & Impl)	系統架構圖 (ADD)、資料庫 ERD、原始程式碼	義大利麵代碼、架構無法承受負載、累積巨額技術債
3. 驗證與確認 (Validation V&V)	自動化測試套件 (PyTest/Jest)、CI 測試報告	產線崩潰當機、交易資料毀損、資安漏洞遭駭客入侵
4. 系統演進 (Evolution)	版本發布說明、資料庫遷移腳本、事故檢討報告	軟體腐化 (Software Rot)、相依套件過期漏洞、系統被迫淘汰

1. Software Specification（需求工程）：

- 透過訪談、情境分析與邊界確認，釐清利害關係人的真實痛點與運作約束。包含需求擷取、衝突分析、規格化撰寫與驗證確認。
- 產出：SRS 規格書、附帶驗收條件 (Acceptance Criteria) 的使用者故事。
- 忽視代價：技術上無懈可擊，但在業務上完全解決了錯誤的問題。

2. Software Design and Implementation（軟體設計與實作）：

- 將抽象需求轉化為可執行的架構與代碼。
- 設計：劃分系統模組層次、定義資料庫關聯模型、制定 API 介面契約、選用設計模式。
- 實作：撰寫高內聚低耦合的乾淨代碼、執行 Peer Code Review、配置建置環境。
- 產出：架構設計文檔 (ADD)、UML 類別/循序圖、資料庫 ERD、原始程式碼。
- 忽視代價：高度緊密耦合的混亂代碼，微小修改即引發系統大崩潰。

3. Software Validation（驗證與確認 V&V）：

- 確保系統符合技術規格（**Verification 驗證**：「我們是否正確地打造了產品？」），並確保系統滿足使用者的真實需求（**Validation 確認**：「我們是否打造了正確的產品？」）。
- 涵蓋單元測試、整合測試、端到端 (E2E) 測試、壓力效能測試、滲透測試與使用者驗收測試 (UAT)。
- 產出：自動化測試套件、持續整合測試報告、缺陷追蹤清單。
- 忽視代價：重大產線事故、客戶信心崩盤、鉅額商業損失。

4. Software Evolution（系統演進與維護）：

- 在軟體交付上線後的長期演進生命週期（更正性維護、適應性維護、完善性維護、預防性維護）。
- 產出：版本 Release Notes、資料庫 Migration 腳本、事後復盤報告 (Post-Mortem)。

- 忽視代價：軟體腐化、難以維護的 Legacy 系統，最終被迫整套報廢。

💡 課堂互動活動 —— 情境分析：哪一項核心活動失敗了？：

情境：一個技術團隊耗時 6 個月，為社區生鮮外送 App 打造了極致流暢、零 Bug 的加密貨幣 (Crypto) 結帳功能。該代碼通過了 100% 的單元測試且產線上零當機。然而上線後發現使用率為 0%，因為該社區長輩顧客只習慣使用「貨到付款」與「實體信用卡」。

問題：四項核心活動中，哪一項發生了根本性失敗？請解釋為何擁有 100% 代碼測試覆蓋率完全無法挽救需求規格 (Specification) 的失敗？

👉 [前往附錄查看情境分析解析](#)

1.6.4 認識軟體工程的多元維度：紀律、原則、方法與啟發準則

為了全面認識軟體工程的廣泛範疇，我們挑選四個實務維度來梳理常見的工程元素（工程實務紀律、核心設計原則、流程方法論與實務啟發準則）：

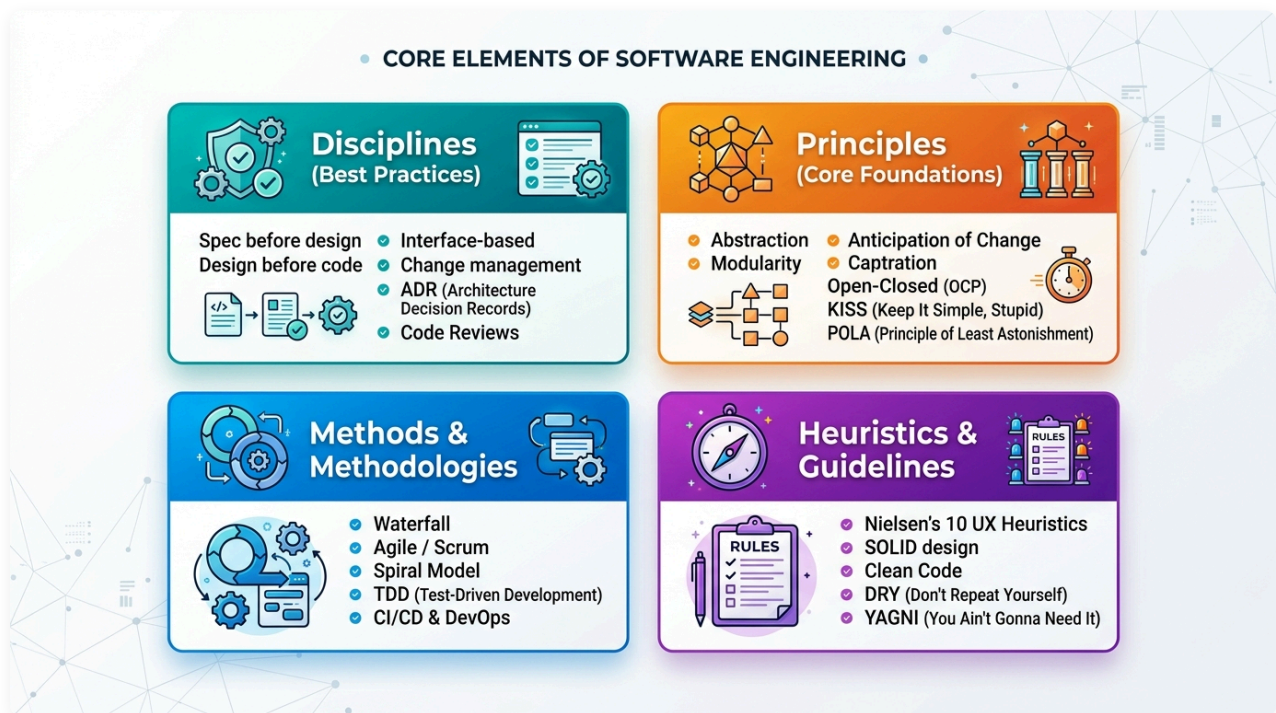


圖 1.6.4：軟體工程多元維度概念架構圖——包含工程實務紀律、核心設計原則、流程方法論與實務啟發準則（下述項目皆為實務具體範例，並非窮舉列舉）。

1. Disciplines（工程實務紀律）

確保團隊具備穩定產出品質的工作紀律與習慣：

- **先規格後設計 (Spec before Design)**：在決定「如何實作」前，必須先徹底釐清「要解決什麼問題」。
- **先設計後編碼 (Design before Code)**：動手寫代碼前先確立模組邊界、介面契約與資料模型。
- **基於介面開發 (Interface-Based Design)**：針對抽象介面（契約）編程，而非綁定具體實作。
- **正式變更管理 (Change Management)**：追蹤並評估每一次需求異動的衝擊，杜絕隨意修改產線代碼。
- **記錄架構決策理由 (ADR)**：詳實記錄為何做出特定技術選擇，為未來維護者保留決策脈絡。
- (更多實務紀律包含：同儕代碼審查 *Peer Code Review*、持續靜態代碼分析、免責復盤演練 *Blameless Post-Mortem...*)

2. Principles（核心設計原則）

指引軟體架構與程式碼設計的基本觀念：

- **Abstraction（抽象化）**：隱藏底層複雜細節，對外提供乾淨的高層介面。
- **Anticipation of Change（預期變更）**：以低耦合度設計架構，預留未來業務變更的彈性空間。
- **Open-Closed Principle（開閉原則 OCP）**：軟體實體應對擴充開放，對修改封閉。
- **KISS（保持簡約，避免過度複雜）**：優先選擇清晰直接的設計，而非炫技卻難懂的過度設計。
- **Principle of Least Astonishment（最小驚訝原則 POLA）**：API 與介面行為應符合直覺，最大程度減少使用者的困惑。
- (更多核心原則包含：職責分離 *Separation of Concerns*、高內聚力 *High Cohesion*、資訊隱藏 *Information Hiding...*)

3. Methods & Methodologies（流程方法論）

組織團隊開發節奏與生命週期管理的具體流程模型：

- **計畫驅動 (Waterfall, V-Model)**：階段嚴密、文件完備，適合高安全性與硬體強綁定的系統。
- **敏捷與 Scrum (Agile & Scrum)**：迭代遞增、快速獲取回饋、強調可運作軟體與跨職能團隊合作。
- **螺旋模型 (Spiral Model)**：以風險驅動為核心，結合原型評估與迭代循環。
- **測試驅動開發 (TDD)**：先寫自動化單元測試再寫功能代碼，驅使架構朝低耦合演進。
- **CI/CD 與 DevOps**：將開發、自動化測試與維運部署深度串接。
- (更多方法論包含：Kanban、極限編程 *XP*、無塵室軟體工程 *Cleanroom...*)

4. Heuristics & Guidelines（實務啟發準則）

數十年工程實務累積的經驗法則與指標：

- **尼爾森 10 大易用性準則 (Nielsen's Heuristics)**：UI 設計的黃金法則（如狀態可見性、防錯設計、辨識優於記憶）。
- **SOLID 原則**：物件導向五大設計原則（單一職責、開閉原則、里氏替換、介面隔離、依賴反轉）。
- **Clean Code 乾淨代碼準則**：具意義命名、單一功能小函式、代碼自帶說明性。

- DRY 原則 (Don't Repeat Yourself)：消除重複邏輯，降低修改時遺漏修復的風險。
- YAGNI (You Aren't Gonna Need It)：不要過早實作未經驗證的推測性需求。
- (更多實務準則包含：迪米特法則 Law of Demeter、童子軍法則 Boy Scout Rule、12-Factor App 原則...)

1.6.5 破除常見的軟體開發迷思

專業軟體工程仰賴客觀事實與工程科學，破除常見的直覺迷思。

- 迷思 1：「專案進度落後了，只要多加幾位工程師就能趕上進度。」
 - 現實（布魯克斯法則 Brooks's Law）：為落後的專案增加人手只會讓進度更加落後。新成員需要資深成員指導，且團隊溝通通道呈平方級增長（ $\frac{n(n-1)}{2}$ ）。
- 迷思 2：「軟體本來就是數位彈性的，後期變更需求成本很低。」
 - 現實：後期修改核心需求會引發資料庫綱要與架構連鎖重構，修改成本可高達初期的 100 倍。

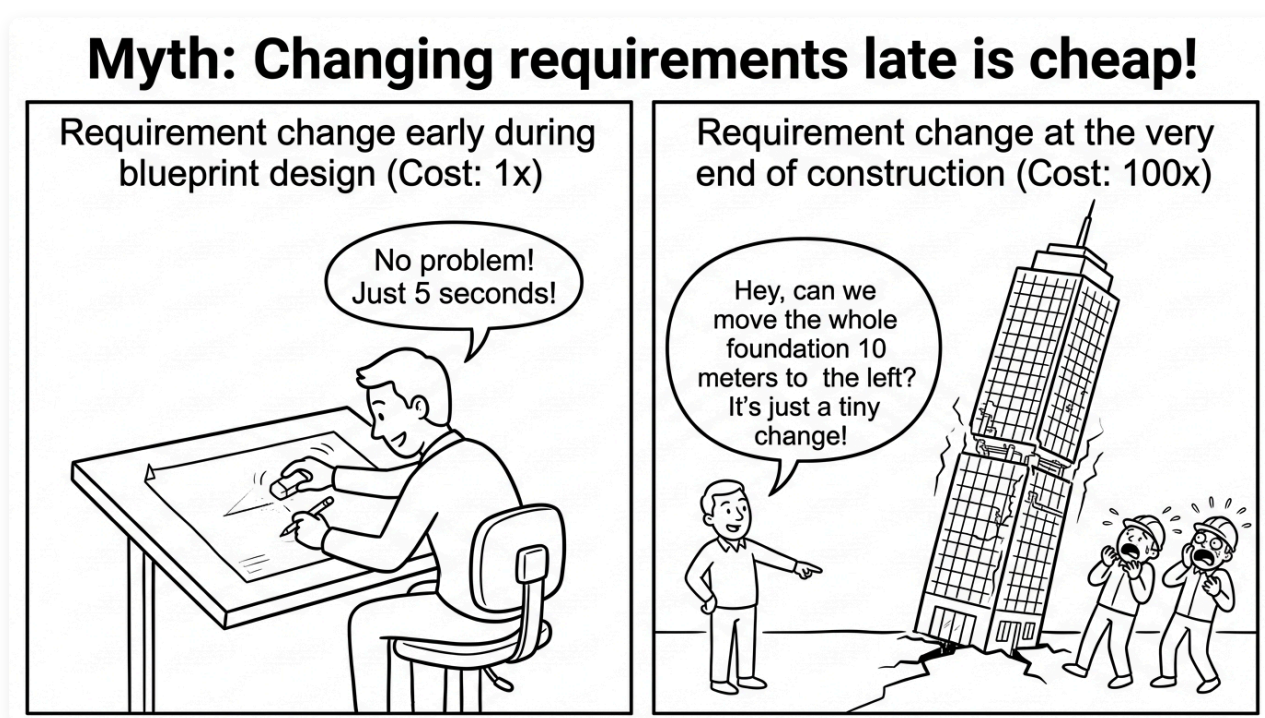


圖 1.6.5：教育漫畫圖解後期變更成本——在藍圖設計初期修改只需橡皮擦擦掉一條鉛筆線（成本 1x），但在 50 層摩天大樓蓋好後才要求「把地基往左移 10 公尺」將導致整棟大樓開裂坍塌（成本 100x）。

- 迷思 3：「把專案外包給廠商後，我們就不需要技術管理了。」
 - 現實：外包若缺乏嚴謹的驗收標準與架構治理，交付的系統往往充斥技術債而無法維護。

💡 🧠 觀念檢驗題 (CCQ 2) 與雙人討論：

問題：專案目前落後 3 週且離上線只剩 2 週，專案經理提議明天立即招聘 4 位新進工程師加快進度。依據布魯克斯法則，最可能的結果為何？

- A) 專案將提早 1 週完成。
- B) 專案將更為延宕，因為資深成員必須停下手邊工作去指導新成員，且溝通成本劇增。
- C) 現有工程師的寫代碼速度將翻倍。

👉 前往附錄查看解答與詳細解析

雙人分享：與身旁同學分享你在過去作業或專案中為了趕死線而留下的「技術債 (Technical Debt)」實例。當時採取了什麼捷徑，後來在修改或擴充時付出了什麼代價？

1.7 現代工具鏈與 AI 雙面刃

現代軟體工程團隊深度仰賴高度自動化的工具鏈與人工智慧來提升開發速度與代碼品質。

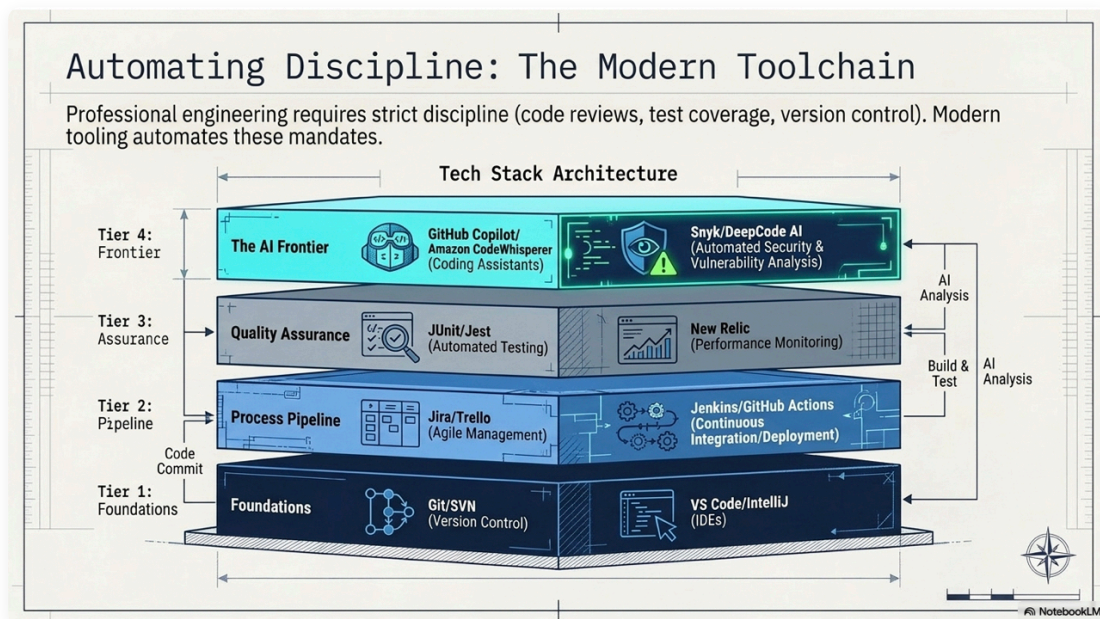


圖 1.7：結構設計模式（如圖中的裝飾者模式 Decorator Pattern）與 CI/CD 自動化工具鏈，是避免系統脆弱與消除人為錯誤的關鍵。

1.7.1 現代工程核心工具鏈

- 版本控制系統 (Git)：分支策略 (Git Flow / Trunk-Based)、Pull Request 協同審查與變更追蹤。
- 現代整合開發環境 (IDEs)：VS Code、IntelliJ IDEA 提供即時靜態分析、智慧重構與斷點除錯。

- **持續整合與持續部署 (CI/CD)**：GitHub Actions、GitLab CI 實現每次 Commit 自動編譯、測試、資安掃描與容器化部署。
- **自動化測試套件**：單元測試 (PyTest/Jest)、整合測試與 E2E 測試 (Playwright)。
- **可觀測性與監控 (Observability & APM)**：Datadog、Prometheus、Sentry 捕獲即時遙測數據與產線 Error Trace。

1.7.2 AI 在軟體工程全生命週期的效益與風險矩陣

大語言模型 (LLM) 與 AI Agent 正在重塑軟體開發。然而負責任的工程師必須深刻理解 AI 在各階段的「效益」與「潛在風險」：

生命週期階段	AI 帶來的核心效益	潛在風險與技術陷阱
1. 需求分析	快速草擬使用者故事、整理 Gherkin 驗收條件	憑空幻覺 (Hallucination) 出虛構業務規則；遺漏隱性組織文化
2. 架構設計	推薦合適架構模式、自動生成 UML 與資料庫 ERD	過度工程化 (Over-engineering)；忽略網路延遲與資安隔離 SLA
3. 程式編程	自動完成樣板代碼、加速演算法撰寫、減少打字疲勞	「盲目相信代碼 (Vibe Coding)」陷阱；引入隱蔽資安漏洞與授權污染
4. 測試驗證	自動生成邊界測試數據、合成測試案例、分析失敗日誌	「回音室測試 (Echo-chamber)」(測試只驗證了 AI 寫出的錯誤邏輯)
5. 維護演進	解讀老舊無文件代碼、自動生成 API 規格書、輔助重構	自動重構引發無感行為退化 (Regression)；累積不易察覺的技術債

 課堂互動活動 —— 隨堂投票與「Vibe Coding」反思：

- **投票：**當你使用 GitHub Copilot、Cursor 或 ChatGPT 產生代碼時，你在按 Commit 前有多常逐行仔細審查並完全理解每行邏輯？ (1: 總是逐行理解 | 2: 大部分時候 | 3: 很少 | 4: 從不，只要能跑就過)
- **討論：**假設 AI 寫出了一段包含 200 行複雜非同步資料庫操作的代碼，且順利通過了 2 個基本測試。這段代碼可以直接推向產線嗎？專業工程師在合併代碼前必須補足哪些驗證紀律？

1.8 專業倫理、社會責任與欺騙性黑暗模式

由於軟體控制著人類生活的方方面面——從醫療呼吸機、航空飛控到金融轉帳與投票系統——軟體工程師對社會承載著巨大的倫理責任。

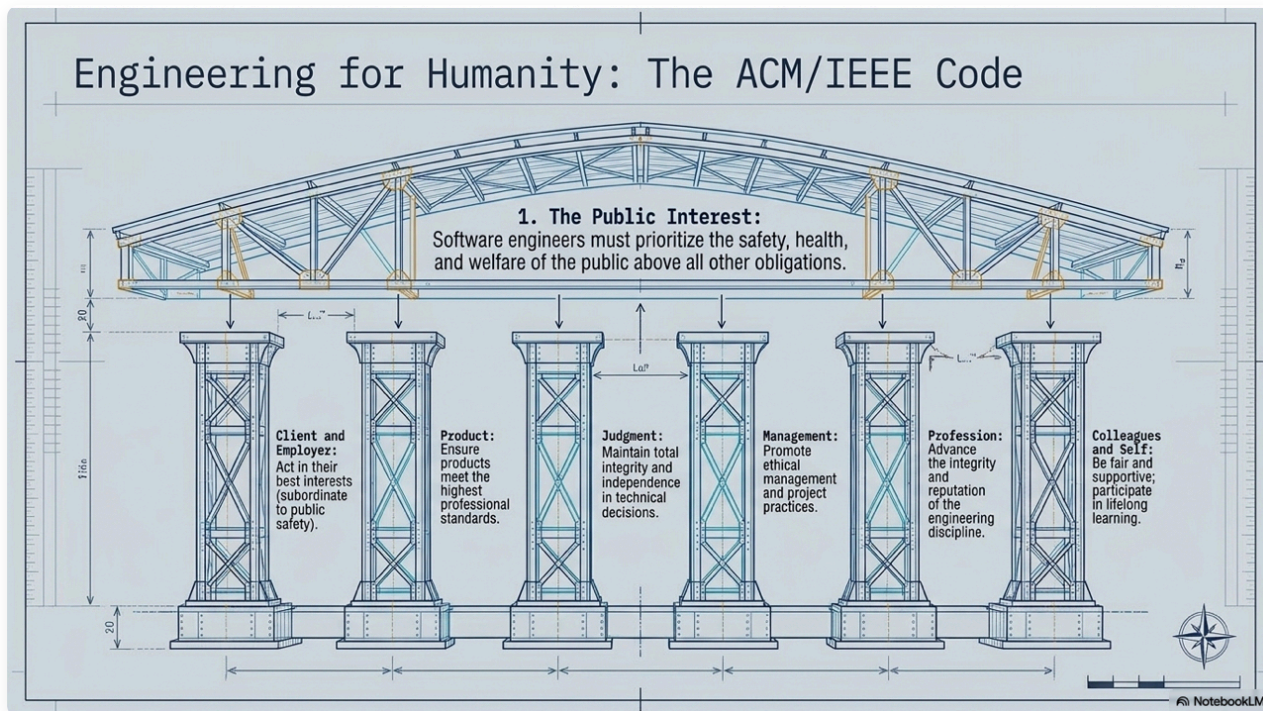


圖 1.8.1：ACM/IEEE 軟體工程倫理守則——確立執業專業人員應遵守的八大核心支柱。

1.8.1 ACM/IEEE 軟體工程倫理守則：八大原則

1. **Public（公眾利益）**：工程師應以公眾利益為最高依歸，將公眾的健康、安全與福祉置於首位。
2. **Client & Employer（客戶與雇主）**：在符合公眾利益的前提下，維護客戶與雇主的最大利益。
3. **Product（產品品質）**：確保產品與後續維護符合最高專業標準。
4. **Judgment（獨立判斷）**：在技術評估與決策中保持誠信與獨立性，拒絕簽署不實聲明。
5. **Management（工程管理）**：主管應推廣倫理管理實務，給予合理的估算與健康的開發環境。
6. **Profession（專業聲譽）**：維護並增進軟體工程專業的誠信與聲譽。
7. **Colleagues（同儕互助）**：公平對待同儕，建立良性指導與回饋文化。
8. **Self（自我提升）**：持續進行終身學習，精進專業技能並推廣倫理實務。

1.8.2 當工程倫理失守：重大歷史教訓

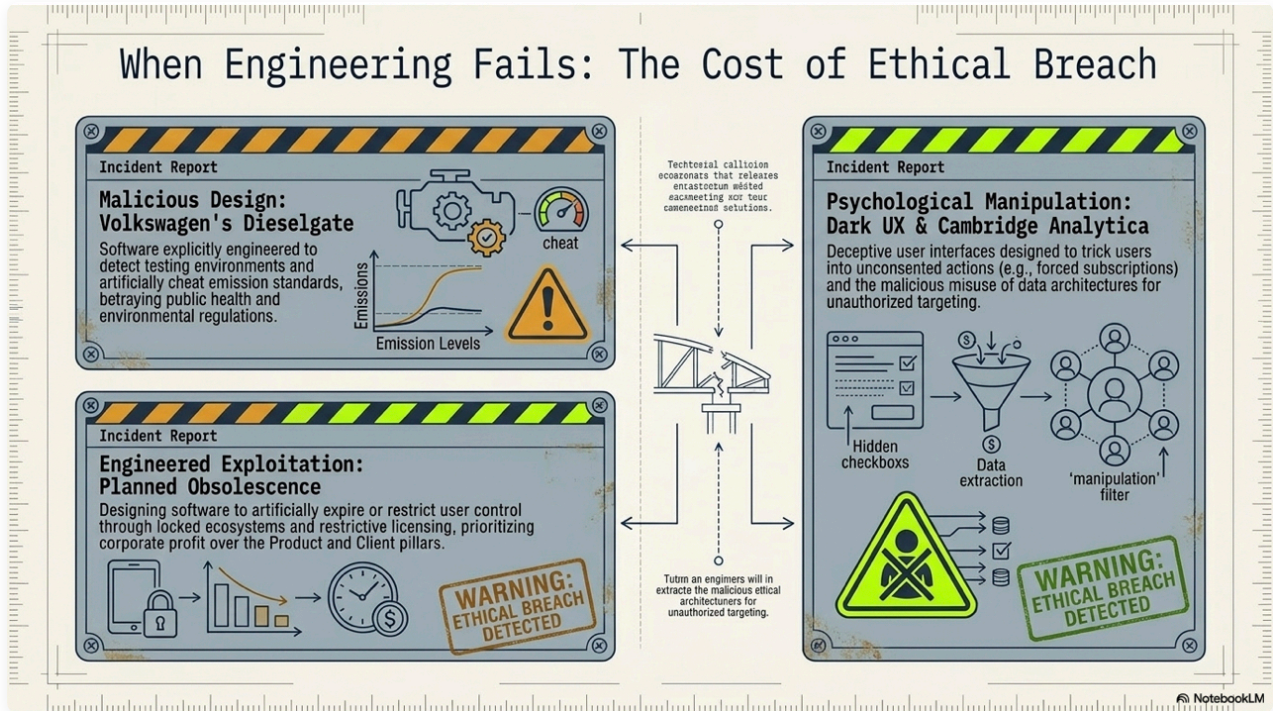


圖 1.8.2：工程倫理失敗案例——揭示當專業誠信遭到踐踏時所帶來的毀滅性社會與環境代價。

- 福斯汽車「柴油門」醜聞 (Volkswagen Dieselgate, 2015) :
 - 工程師依指示撰寫作弊軟體，當偵測到車輛處於實驗室檢驗狀態時自動減少廢氣排放；但在實際道路行駛時排放高達法定上限 40 倍的 toxic 氮氧化物 (NO_x)。
 - 導致數百億美元罰款、主管刑事定罪與嚴重的環境公害。
- 劍橋分析事件 (Cambridge Analytica, 2018) :
 - 不當收集與濫用數千萬使用者的社群隱私數據進行政治操控，凸顯了「隱私設計 (Privacy by Design)」的極端重要性。
- 計畫性報廢 (Planned Obsolescence) :
 - 透過軟體更新蓄意降低舊款硬體的效能或阻斷相容性，強迫消費者購買新機。

1.8.3 欺騙性「黑暗模式」(Dark Patterns) 漫畫解析

Dark Pattern (黑暗模式) 是一種經過精心設計的使用者介面，旨在誘騙使用者做出違背其本意的決策——例如購買非自願的保險、自動續訂高額月費、或交出個人隱私。

DECEPTIVE UI/UX DARK PATTERNS

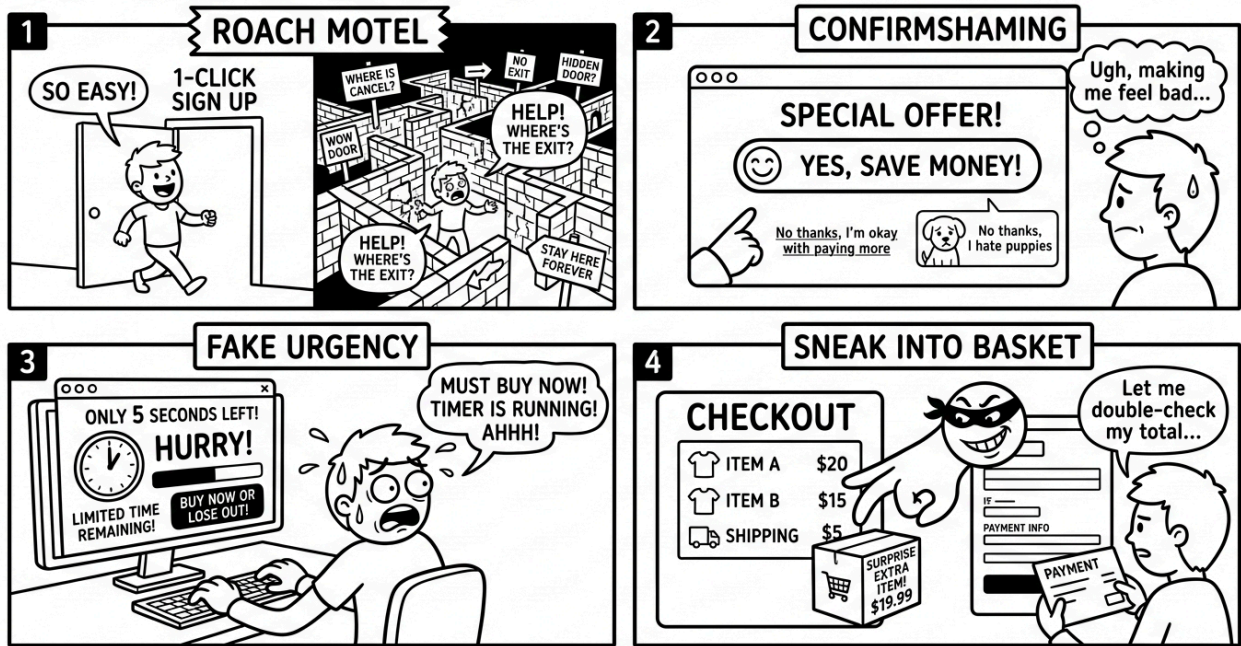


圖 1.8.3：純黑白線條四格漫畫揭露常見的四大欺騙性黑暗模式——1. Roach Motel、2. Confirmshaming、3. Fake Urgency 與 4. Sneak into Basket。

四大典型欺騙性黑暗模式：

- 1. Roach Motel (蟑螂屋 / 訂閱迷宮)**：1 鍵快速訂閱極為容易，但想要取消訂閱時，卻隱藏在錯綜複雜的層層選單或強迫撥打客服電話。
- 2. Confirmshaming (羞辱性確認)**：在拒絕按鈕上使用引發內疚或羞恥感的文案（例如大按鈕寫著「是的，我要享受優惠！」，下方灰色小字寫著「不要，我討厭省錢與小狗」）。
- 3. Hidden Costs & Sneak into Basket (偷塞購物車)**：在結帳最後一步前，系統自動預先勾選附加保險或偷加手續費。
- 4. Fake Urgency (製造虛假緊迫感)**：畫面上出現虛假的倒數計時器（「優惠只剩 5 秒鐘！」）或虛構警示（「目前有 34 人正在瀏覽此房間」），逼迫使用者衝動下單。

專業工程師的使命：專業軟體工程師應堅決拒絕實作誤導性的黑暗模式，積極倡導透明、誠實且尊重使用者的軟體體驗。

The Mindset of a Software Engineer

Master Architect's Sign-Off

SIGNATURE BLOCK 1: SYSTEM HOLISM Beyond the Code: Software is not just algorithms; it is a complete system of programs, procedures, documentation, and data. SIGNATURE: _____ DATE: _____ ENGINEER'S SEAL _____	APPROVAL STAMP 	STATUS: VERIFIED INSPECTOR: CHIEF ARCHITECT
SIGNATURE BLOCK 2: QUALITY ASSURANCE Beyond "Working": True quality demands usability, dependability, efficiency, and maintainability—not just conforming to basic requirements. SIGNATURE: _____ DATE: _____ ENGINEER'S SEAL _____	APPROVAL STAMP 	STATUS: COMPLIANT INSPECTOR: QA LEAD
SIGNATURE BLOCK 3: PRACTICAL REALITIES Beyond Theory: Unlike theoretical computer science, Software Engineering focuses on the practical development of useful systems under strict real-world constraints (time, budget, human capital). SIGNATURE: _____ DATE: _____ ENGINEER'S SEAL _____	APPROVAL STAMP 	STATUS: VALIDATED INSPECTOR: PROJECT MANAGER
SIGNATURE BLOCK 4: ETHICAL OBLIGATION Beyond the Screen: The ultimate metric of a software engineer's success is the protection and advancement of the public interest. SIGNATURE: _____ DATE: _____ ENGINEER'S SEAL _____	APPROVAL STAMP 	STATUS: APPROVED INSPECTOR: ETHICS OFFICER

NotebookLM

圖 1.8.4：軟體工程師的專業心態——深刻體會我們對資料真實性、系統穩定性與使用者信任的崇高責任。

💡 課堂互動活動 —— 黑暗模式偵探：

- **調查**：回想你最近在 App 或網站上遇過的一個欺騙性黑暗模式。
- **剖析**：該設計違反了 ACM/IEEE 倫理守則中的哪一項原則（公眾利益、產品品質、或專業判斷）？
- **重構**：你會如何重新設計該介面流程，使其既能兼顧合理的商業轉化，又能保持完全的誠信與透明？

1.9 軟體工程常見問答 (FAQ)

1.9.1 核心差異與成本現實

Q1：電腦科學 (Computer Science) 與軟體工程 (Software Engineering) 的核心差異為何？

- **電腦科學**：專注於計算理論與數學基礎——如演算法時間複雜度、資料結構、編譯器理論與自動機。
- **軟體工程**：專注於實務應用工程——在時間、預算與團隊協作等現實約束下，設計、建置、測試與維護可靠的軟體系統。

Q2：軟體生命週期的成本如何分佈？

- 在初始開發階段，約 60% 成本用於開發（需求、架構、編程），40% 成本用於驗證與測試。
- 在整個長期的軟體生命週期中，上線後的維護與演進 (Maintenance & Evolution) 往往佔據總成本的 70% 到 80%。

Q3：是否存在一套適用於所有軟體專案的「最佳方法論」或「最強程式語言」？

- 不存在。不同系統受制於完全不同的現實約束。AAA 級遊戲需要極致的圖形效能與快速原型迭代 (C++/Unreal)；飛機飛控系統需要形式化嚴格驗證與確定性執行 (Ada/Rust)；網路新創團隊追求快速市場驗證與即時迭代 (TypeScript/Python)。專業軟體工程的核心在於為特定問題挑選最適當的工具與流程。

附錄：互動活動解答與詳細解析

CCQ 1 —— 軟體危機的本質

- **正確答案：**B（危機本質上是人類心智面對架構複雜度、跨人溝通成本與缺乏規範的認知危機，更快的硬體只會放大問題規模）。
- **詳細解析：**硬體運算能力的爆發允許人類開發前所未有的大型系統。然而，若軟體開發仍停留在個人隨性編程的手工作法，團隊面對龐大系統時將無法掌握相依性與介面契約，導致複雜度超出人類大腦所能掌控的極限。更快的 CPU 並無法自動解決需求模糊、架構混亂或缺乏測試紀律的問題。
- [↑ 返回第 1.2 節](#)

情境分析：哪一項核心活動失敗了？

- **正確答案：**需求規格 (Specification) 發生了根本性失敗。
- **詳細解析：**該團隊達成了 100% 的測試覆蓋率與產線零當機（**Verification 驗證：**「我們是否正確地打造了產品？」→ 是）。然而，他們打造的功能完全不符合使用者的真實需求（**Validation 確認：**「我們是否打造了正確的產品？」→ 否）。因為前期忽略了利害關係人訪談與市場痛點分析，導致耗費 6 個月開發出無人使用的錯誤功能。代碼測試覆蓋率無法彌補需求規格的錯誤。
- [↑ 返回第 1.6.3 節](#)

CCQ 2 —— 布魯克斯法則與專案動態

- **正確答案：**B（專案將更為延宕，因為資深成員必須停下手邊工作去指導新成員，且溝通成本劇增）。
- **詳細解析：**布魯克斯在《人月神話》中指出，複雜軟體專案無法像體力勞動一樣簡單線性切割。在專案緊迫的收尾階段加入新人，會導致現有資深工程師必須抽出大量時間進行教育培

訓，且團隊人與人之間的溝通通道數以 $\frac{n(n-1)}{2}$ 平方級暴增，進一步拖慢進度。

- [↑ 返回第 1.6.5 節](#)

參考文獻與延伸閱讀

- Sommerville, Ian. *Software Engineering* (10th Edition). Pearson. [官方網站](#)
- Brooks, Frederick P. *人月神話 (The Mythical Man-Month)*.
- ISO/IEC 9126-1:2001. *Software engineering — Product quality — Part 1: Quality model*.
- ACM/IEEE 軟體工程倫理與專業實務守則. [IEEE CS 官網](#)
- Brignull, Harry. *Deceptive Patterns: Exposing the Tricks Tech Companies Use to Control You*.